De l'invariant à l'interface.

Construire et expliquer LedgerX, étape par étape

Un carnet de construction pour comprendre les décisions, relier le code aux preuves et préparer une démonstration technique crédible.

LE FIL CONDUCTEUR

Un mouvement d'argent ne doit jamais exister à moitié.

Transactions atomiques, double entrée, verrous en base, idempotence et journal immuable forment le socle. L'interface vient rendre ce socle utilisable.

01 / COMPRENDRE

Audit de l'existant, périmètre et invariants.

02 / CONSTRUIRE

Transactions, sécurité, remboursements et contrôles.

03 / PROUVER

PostgreSQL réel, concurrence, mesures et limites.

04 / RENDRE UTILISABLE

React, montants exacts, parcours et documentation.

NOTE DE MÉTHODE

Le projet existait à la reprise. Ce document reconstruit l'ordre logique des travaux ; il ne prétend pas être un journal Git horodaté ni un travail sans assistance IA.

Comprendre avant d'étendre.

Étapes 01 et 02 · Partir du dépôt existant et préserver ses garanties financières.

01 Inspecter le projet existant

ACTION

Lecture du README, des routes FastAPI, services, modèles SQLAlchemy, migrations Alembic, scripts et tests. Repérage des limites de l'environnement et conservation du monolithe modulaire.

POURQUOI

Réécrire le dépôt aurait rendu plus difficile la préservation des invariants. Il fallait d'abord identifier qui ouvre la transaction, qui verrouille les comptes et qui fait respecter la comptabilité.

PREUVE / LIMITE

Repères : app/api, app/services, app/db, alembic/versions et tests. Le dépôt n'avait pas d'historique Git disponible pour attribuer chaque ligne antérieure.

02 Fixer les invariants de l'argent

ACTION

Montants en centimes entiers ; un transfert produit deux écritures opposées ; les comptes clients ne peuvent pas devenir négatifs ; le journal ne peut être ni modifié ni supprimé.

POURQUOI

Le solde est une vue matérialisée du journal. Les deux doivent rester cohérents. La base vérifie les contraintes financières même si une nouvelle route ou un autre client oublie un contrôle.

PREUVE / LIMITE

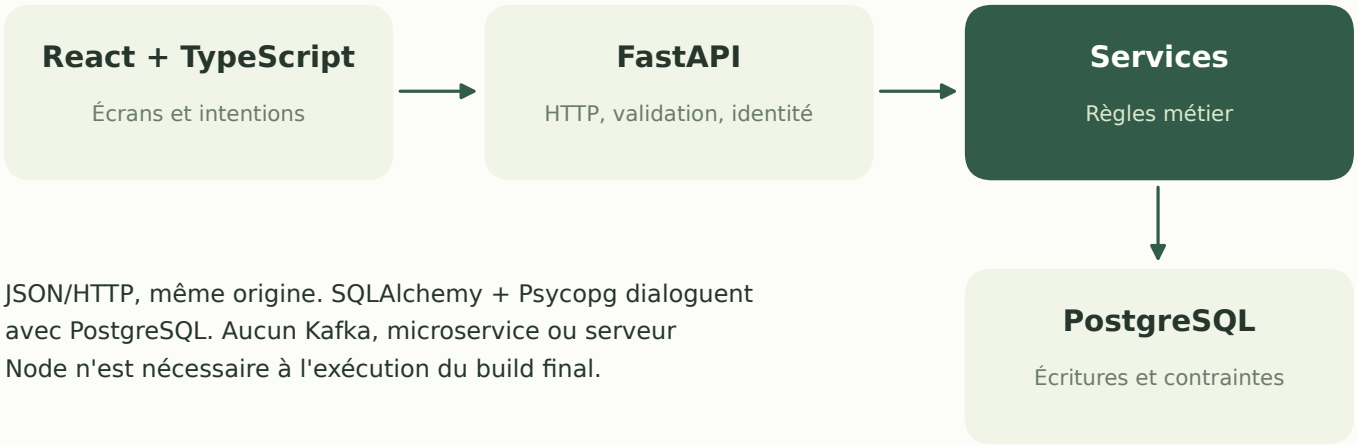
Compte système réservé au financement fictif. EUR uniquement. Plafond de 9 × 10^18 centimes. Contraintes, déclencheurs différés et tests d'intégrité dans PostgreSQL.

À SAVOIR EXPLIQUER

Pourquoi un float n'est pas adapté à l'argent ; pourquoi une validation Python seule ne protège pas tous les chemins d'écriture.

Un monolithe, des frontières.

Étapes 03 et 04 · Responsabilités séparées, transaction financière unique.



JSON/HTTP, même origine. SQLAlchemy + Psycopg dialoguent avec PostgreSQL. Aucun Kafka, microservice ou serveur Node n'est nécessaire à l'exécution du build final.

03 Conserver une séparation simple

ACTION

Les schémas Pydantic portent les contrats ; les routes gèrent HTTP et les transactions ; les services exécutent le métier ; les modèles et migrations définissent la persistance.

POURQUOI

L'OOP sert aux entités, sessions et contrats. Les services restent des fonctions explicites : aucune hiérarchie de classes artificielle n'est nécessaire pour déplacer de l'argent.

PREUVE / LIMITE

Repères : app/schemas, app/api, app/services et app/db. Le frontend appelle les mêmes routes documentées dans /docs.

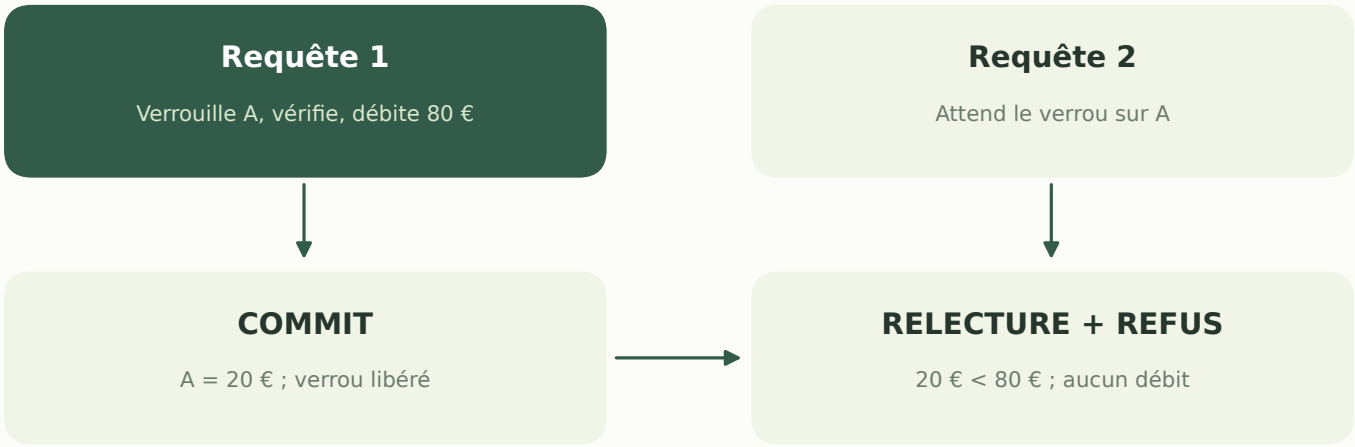
04 / UNE FRONTIÈRE ATOMIQUE

La route ouvre session.begin(). Le service réserve la clé, verrouille les comptes, vérifie les fonds, inscrit le transfert et ses deux écritures. Un seul commit rend tout visible ; toute exception annule l'ensemble.

Deux clients. Un même solde.

Étapes 05 et 06 · Traiter les courses avec les garanties de PostgreSQL.

EXEMPLE : A DISPOSE DE 100 € ; DEUX DEMANDES VEULENT DÉBITER 80 €



05 Verrouiller dans un ordre stable

ACTION

Les deux comptes sont verrouillés par UUID croissant avec SELECT FOR UPDATE. Tous les chemins financiers suivent le même ordre pour les comptes.

POURQUOI

Cela sérialise les dépenses sur une même source et réduit les risques d'interblocage entre virements opposés. Un mutex Python ne coordonnerait pas plusieurs processus API.

PREUVE / LIMITE

Tests de concurrence avec plusieurs connexions PostgreSQL. La sûreté n'élimine pas la contention : les virements partageant une source ralentissent sous forte concurrence.

06 / RAFRAÎCHIR AUSSI L'IDENTITY MAP

Un verrou SQL ne remplace pas automatiquement un objet déjà présent dans la Session SQLAlchemy. populate_existing=True recharge le solde verrouillé ; un test de régression protège ce cas subtil.

Une intention, un effet.

Étapes 07 et 08 · Répétition réseau, nouvelle opération et propriétaire.

07 Rendre les répétitions sûres

ACTION

La clé d'idempotence est indexée par utilisateur et type d'opération. Une empreinte SHA-256 du contenu détecte les réutilisations incompatibles. La réponse d'un transfert terminé est retrouvée en base.

POURQUOI

Une réponse HTTP perdue ne signifie pas un paiement annulé. Le client peut réémettre la même intention, avec le même contenu et la même clé, sans effectuer un second débit.

PREUVE / LIMITE

Optimisation introduite : INSERT ... RETURNING récupère directement une nouvelle réservation ; le chemin de conflit conserve sa lecture verrouillée. Ne pas attribuer les anciennes mesures à cette version sans la remesurer.

08 Ajouter identité et propriété

ACTION

Mots de passe dérivés avec scrypt et sel aléatoire ; tokens opaques aléatoires dont seul le condensat est conservé en base ; expiration et révocation. Chaque compte appartient à un utilisateur.

POURQUOI

L'authentification identifie ; l'autorisation décide. Connaître un UUID ou une clé d'idempotence ne doit pas permettre d'accéder aux comptes d'un autre utilisateur, même avec un rôle administrateur.

PREUVE / LIMITE

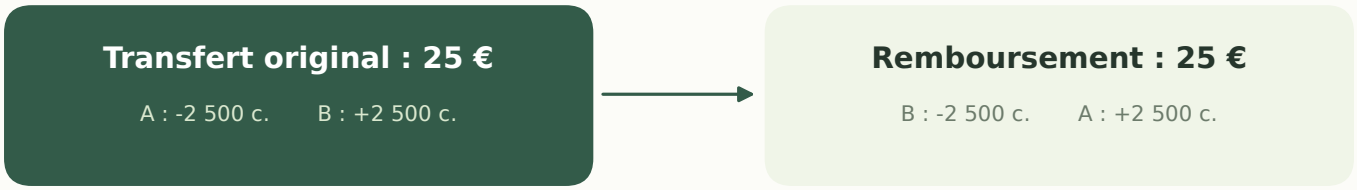
Tests d'absence de token, de tokens expirés et révoqués, de rôles et d'accès croisés. Le financement reste administratif et soumis à un flag explicite. Aucun MFA ni mécanisme de récupération de compte à ce stade.

LE POINT D'ENTRETIEN

Les tokens du navigateur restent en mémoire. Un rechargement déconnecte ; aucun bearer token n'est persisté dans localStorage.

Corriger sans effacer.

Étapes 09 et 10 · Compensation et contrôle indépendant du journal.



Lien reversal_of · les anciennes écritures restent intactes

09 Créer un transfert compensatoire

ACTION

Le destinataire d'origine peut retourner l'intégralité du paiement. La ligne d'origine est verrouillée pour sérialiser les demandes ; une contrainte unique interdit une seconde compensation.

POURQUOI

Une annulation destructive détruirait l'audit. Une nouvelle écriture conserve la cause et l'effet, vérifie les fonds disponibles et respecte encore la double entrée. Le verrou NO KEY UPDATE évite un conflit inutile avec les verrous de clés étrangères.

PREUVE / LIMITE

Remboursements complets uniquement, idempotents ; financement et remboursement d'un remboursement interdits. Tests de tentatives simultanées, de propriétaire et de solde insuffisant.

10 Recalculer indépendamment

ACTION

Le moteur de réconciliation somme les écritures de chaque compte et compare le résultat au solde stocké. Il vérifie aussi la structure des transferts et les totaux du journal dans un même instantané SQL.

POURQUOI

Le contrôle ne doit pas simplement réutiliser le résultat du service de virement : il confronte deux représentations indépendantes des montants.

PREUVE / LIMITE

Endpoint administratif ; rapport structuré ; aucune réparation silencieuse. Une corruption volontaire et isolée de sept centimes vérifie que l'écart est détecté.

Chercher les pannes utiles.

Étapes 11 et 12 · Observer sans divulguer, puis tester les garanties réelles.

11 Instrumenter les opérations

ACTION

Ajout de logs JSON avec identifiants de requête et de corrélation, métriques Prometheus et mesures de latence. Les erreurs SQL sont converties en réponses expurgées sans paramètres sensibles.

POURQUOI

On doit suivre un incident sans exposer mots de passe, tokens ou données financières dans les logs. Les labels de métriques sont bornés : pas d'identifiant de compte par série.

PREUVE / LIMITE

Injection de panne après modification en mémoire et après flush ; validation du rollback. Tests de lock timeout et de filtrage des exceptions inattendues. Timeouts de connexion, pool, verrou et requête explicités.

12 Tester sur PostgreSQL réel

ACTION

Tests unitaires, intégration avec plusieurs connexions, tests HTTP, migrations upgrade/downgrade lorsque réversibles et vérification de dérive ORM. Bases destructibles protégées par des suffixes dédiés.

POURQUOI

SQLite ne reproduit pas les verrous ni les déclencheurs PostgreSQL. Les tests financiers doivent vérifier aussi l'état final : soldes, écritures, comptes et idempotence après un échec.

PREUVE / LIMITE

Version 0004 validée : 147 tests unitaires/intégration et 2 E2E HTTP, 94,12 % de couverture. Rapports archivés dans artifacts/releases. La CI distante reste à exécuter ; le déploiement Docker local réussit.

RÈGLE DE PRÉSENTATION

Un test écrit n'est pas un test exécuté. Une couverture élevée n'est pas une preuve d'absence de bugs ; elle complète les assertions métier.

Mesurer avant de promettre.

Étapes 13 et 14 · Référence du 20 octobre ; résultats de la version 0004 en page 14.

13 Construire une matrice de charge

ACTION

Dix scénarios HTTP, 1, 8 et 32 clients en concurrence, trois répétitions : 90 cas et 18 000 requêtes chronométrées. Les scénarios distinguent lectures, virements indépendants, source partagée, répétitions et remboursements.

POURQUOI

Une moyenne globale masquerait l'attente des verrous et les queues de latence. On conserve les données brutes, les percentiles et les contrôles de cohérence après chaque scénario.

PREUVE / LIMITE

Référence locale : tous les cas avaient les réponses attendues et une réconciliation cohérente. Les réponses d'insuffisance de fonds attendues sont des succès du scénario de test, pas des virements réussis.

Scénario / 8 clients	Requêtes/s	p95
Lecture du solde	926,5	10,44 ms
Virements indépendants	443,1	21,44 ms
Source partagée	365,5	56,52 ms

Médianes des 3 répétitions, y compris médiane des p95 ; pas un p95 agrégé.

14 Profiler et conserver la référence

ACTION

cProfile, comptage SQL, EXPLAIN ANALYZE et notebook avec courbes. La référence est archivée avant l'optimisation de la réservation d'idempotence, qui vise à supprimer une lecture SQL redondante.

POURQUOI

La profondeur du journal, la pagination et le niveau de concurrence n'ont pas les mêmes coûts. Un plan SQL plus rapide ne prouve pas automatiquement une amélioration HTTP de bout en bout.

PREUVE / LIMITE

Référence : 7 appels SQL par transfert ; 6 observés sur la version 0004. Ces comptes excluent les instructions internes aux triggers. 32 clients testés ne donnent pas une capacité commerciale garantie.

Rendre le système utilisable.

Étapes 15 et 16 · Construire une interface originale sur les contrats existants.

15 Ajouter React sans refaire l'API

ACTION

React et TypeScript construisent les écrans ; Vite produit un bundle statique servi par FastAPI. Deux lectures protégées sont ajoutées : /auth/me et la liste paginée /accounts, filtrée par propriétaire.

POURQUOI

Le dashboard utilise les comptes réels et leurs écritures. Les graphiques indiquent leur fenêtre et leur périmètre chargé ; aucune fausse dépense ou banque externe n'est injectée dans le produit.

PREUVE / LIMITE

Connexion, création de compte, réception par UUID, virement avec confirmation, historique, détail, remboursement et réconciliation admin. CSS responsive original, dialogue natif, navigation clavier et réduction des animations.

16 Préserver la précision et l'intention

ACTION

Les centimes restent des BigInt dans le navigateur. lossless-json préserve les entiers au passage JSON. Une intention de paiement fige la charge utile et la clé d'idempotence pour les nouvelles tentatives.

POURQUOI

JSON.parse avec Number peut arrondir un grand entier avant même son affichage. Une panne réseau peut survenir après le commit : le bouton Réessayer doit réutiliser l'intention au lieu d'en créer une autre.

PREUVE / LIMITE

26 tests frontend exécutés à cette étape couvrent conversions, bornes et reprises. La compilation TypeScript + Vite réussit. Les tests navigateur sont distingués entre API simulée et parcours sur PostgreSQL réel.

LIMITES ASSUMÉES

La session et la clé en attente restent en mémoire. Après rechargement ou expiration, vérifier l'historique avant de créer un nouveau paiement.

Pouvoir le refaire. Et l'expliquer.

Étapes 17 et 18 · Reproductibilité locale et présentation honnête des résultats.

17 Rendre l'installation reproductible

ACTION

Scripts dédiés pour Node local, npm, compilation, tests navigateur et vérification backend. uv.lock et package-lock.json enregistrent les dépendances. Caches et téléchargements restent dans le projet.

POURQUOI

Un reviewer doit pouvoir suivre quelques commandes, sans compte cloud, et comprendre ce qui démarre. FastAPI sert le frontend compilé ; Node n'est nécessaire qu'au build et au développement frontend.

PREUVE / LIMITE

scripts/verify_local.sh utilise _test et _bench ; scripts/verify_web.sh utilise _bench.
Le README précise chaque commande. Ne pas commettre node_modules, .venv, .runtime ni les secrets.

18 Préparer une démo vérifiable

ACTION

Démontrer deux comptes, un financement fictif administrateur, un virement, un remboursement et une réconciliation. Relier chaque écran aux services et à un test pertinent, puis présenter les mesures archivées.

POURQUOI

Le projet est convaincant lorsqu'on sait expliquer un compromis, reproduire un résultat et reconnaître une limite. Ajouter des infrastructures ou des features décoratives ne remplace pas cette maîtrise.

PREUVE / LIMITE

Aucun argent réel, aucune conformité bancaire revendiquée. TLS public, sauvegarde/restauration, quotas, protection distribuée, surveillance et charge longue restent à traiter avant une ouverture à de vrais utilisateurs.

PROVENANCE ET NOM

Travail accompagné par IA ; revue et compréhension personnelles nécessaires. LedgerX est un nom de travail déjà utilisé ailleurs ; le projet est indépendant et non affilié à Revolut ou à Ledger.

Du récit aux preuves.

Les fichiers à consulter pour vérifier les explications et reproduire les parcours.

01 Architecture et décisions
README.md · docs/ENGINEERING_REPORT.md

02 Invariants et schéma
app/db/models.py · alembic/versions/

03 Transactions et verrous
app/services/transfer_service.py

04 Idempotence et identité
app/services/idempotency_service.py · auth_service.py

05 Réconciliation
app/services/reconciliation_service.py

06 Preuves PostgreSQL
tests/ · artifacts/integration.xml · artifacts/e2e.xml

07 Mesures et référence
artifacts/latency/ · artifacts/baseline/

08 Analyse visuelle
analysis/ledgerx_performance.ipynb

09 Interface et contrats
frontend/src/ · frontend/tests/ · frontend/e2e/

10 Résultats navigateur
artifacts/frontend-browser-results.json

11 Guide de démonstration
docs/DEMO.md

ÉTAT DU RAPPORT AU MOMENT DE LA GÉNÉRATION

Version 0004 vérifiée : 12 réussis, 0 en échec, 0 ignorés. Deux parcours réels dans Docker ; déploiement local terminé le 22 octobre.

De la démo locale au web.

Étapes 19 et 20 · Docker validé sur 0004 ; partage HTTPS à tester.



Accès public futur : tunnel HTTPS ou hébergement distant.

19 Borner le coût de l'authentification

ACTION

Connexions et inscriptions partagent une fenêtre glissante et deux places simultanées. Une deque, des compteurs et un verrou protègent les réservations en mémoire.

POURQUOI

Le hachage scrypt coûte du CPU et de la mémoire. Une requête refusée reçoit HTTP 429 et Retry-After avant la vérification du mot de passe.

PREUVE / LIMITE

Six tests dédiés passent. Limites locales au processus, remises à zéro au redémarrage ; plusieurs visiteurs derrière un proxy peuvent partager le même quota.

20 Préparer et vérifier le conteneur public

ACTION

Dépendances Python verrouillées, PORT fourni par l'hébergeur, arrêt si migration en échec. La dernière étape Docker produit l'application, pas l'image de tests.

POURQUOI

La préparation distingue la base locale de la base distante. Le scénario Docker isolé transfère puis rembourse entre deux vrais profils clients.

PREUVE / LIMITE

Version 0004 : 149 tests backend, 26 frontend et 12 navigateur passent. Sauvegarde créée et app locale migrée. Restauration non encore testée.

HISTORIQUE GITHUB

Importer honnêtement le travail local, puis créer des PR pour les évolutions réelles. Les résultats et dates correspondent aux vérifications exécutées.

Retirer le coût de l'historique.

Étapes 21 et 22 · Projection incrémentale et charge à arrivées fixes.



21 Maintenir le total du journal à chaque insertion

ACTION

Le trigger additionne les nouvelles écritures au lieu de relire tout l'historique. Il protège la projection ; les contraintes de double entrée restent.

POURQUOI

Le coût de validation ne doit pas augmenter avec l'âge du compte. La réconciliation recalcule encore le journal complet indépendamment.

PREUVE / LIMITE

Dix tests PostgreSQL ajoutés : falsification, lots, concurrence, rollback et migration avec fonds. Tous passent dans la suite PostgreSQL sur le Mac.

22 Mesurer la latence sans masquer la saturation

ACTION

Arrivées programmées à débit fixé, tâches simultanées bornées. Chaque perte est comptée ; le délai de programmation entre dans la latence.

POURQUOI

En boucle fermée, un serveur lent ralentit aussi le générateur. La nouvelle mesure distingue débit offert, réponses, pertes, p95 et p99.

PREUVE / LIMITE

18 cas mesurés : comptabilité cohérente, objectifs de charge élevés non tenus. Les graphiques exposent les abandons du générateur, sans les masquer.

MESURE ET COMPROMIS

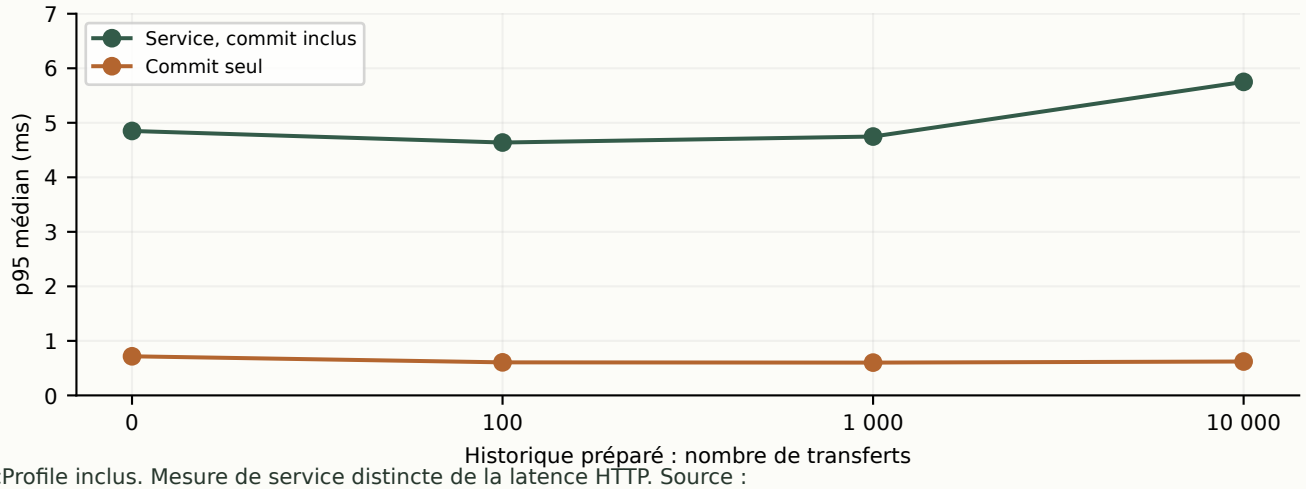
Projection : mises à jour et WAL supplémentaires. Verrous toujours nécessaires. Les anciens chiffres restent une référence archivée, pas une preuve de gain.

Les preuves du 22 octobre.

Étapes 23 et 24 · Mesures réelles, limites explicites et app disponible sur le Mac.

149	26	12
tests backend	tests frontend	tests navigateur

PROFILAGE : 3 RÉPÉTITIONS DE 100 TRANSFERTS PAR PROFONDEUR



cProfile inclus. Mesure de service distincte de la latence HTTP. Source : artifacts/latency/profiling.json.

23 / CONSERVER LES LIMITES DE CHARGE

30 000 arrivées proposées : 18 439 envoyées, 11 561 abandonnées par le générateur. Aucune réponse inattendue ; rapprochements cohérents. Les six essais courts à 100 req/s passent, mais seulement deux sur six à 300 et aucun à 600. Aucun SLO garanti.

24 / LIVRER ET PARTAGER

Image Docker construite, base personnelle sauvegardée, migration 0004 et contrôles de disponibilité réussis. Utilisation : http://127.0.0.1:8000. Le tunnel HTTPS temporaire permet à un ami de s'inscrire pendant que le Mac reste allumé ; pas besoin de compte Cloudflare.

CE QUI RESTE À VÉRIFIER

Parcours navigateur externe à terminer, CI GitHub à lancer, restauration sur base séparée et test de charge longue à faire. La sauvegarde est créée, sa restauration n'a pas encore été vérifiée.